NTNU
Kunnskap for en bedre verden

Department of Computer Science

IT3708 - Bio-Inspired Artificial Intelligence

Genetic Island Algorithm

Project 2

Authors :
Johannes Lorentzen and Svein Kåre Sørestad

March, 2026

1 Introduction

This report details the problem description, methodology and the resulting solutions to a kind of vehicle routing problem (VRP) with nurses to home care patients. In the problem section, the goal and constraints of the problem will be explained in detail. The methodology chapter describes the trials and errors made under this project, and how the algorithm was developed and evaluated. Finally, the results are presented, followed by a discussion about choices made and a short conclusion.

2 Problem

The problem, as presented by Visma, was a VRP where a set of patients was to be assigned to a set of nurses. Each patients had a time window, demand, and care time associated with them. Additionally, a matrix containing all the travel times was provided. The goal of the project was to minimize the total travel time of all the nurses, ensuring that no constraint was violated.

The constraints for the problem were focused around time and demand. All the nurses had to return to the depot before the specified return time. Also, each nurse had to finish their care of a patient within the patients time window, although the nurse could wait if they arrived before the time window started without violating a constraint. Waiting and care time were not included in travel time. Additionally, each nurse had a capacity, and this capacity could not be exceeded. Finally, each patient had to be assigned exactly once.

3 Methodology

This section covers the design of the algorithm used to solve the test instances. The algorithm was a "ruin and repair" approach, where multiple islands were used to increase diversity. Since the goal was to minimize travel time, the algorithm was implemented in a way that minimized fitness.

3.1 Representation

The first major choice for this problem was the genotype representation. In the previous project, a simple binary string was sufficient. However, due to the complex nature of the problem, a more sophisticated representation was needed. The group decided that a vector of integers would be most appropriate, where a 0 would mark a depot. Since each patient was numbered starting at 1, this was a simple and efficient way of representing them. Since each route had to start and end at a depot, 0 also worked as the route separator. This choice was also influenced by the fact that the depot was index 0 in the travel times matrix, which means that this integer representation could be used directly to look up travel times. An example of this representation is shown in equation 1, where the patients [1, 4] are divided into 3 routes.

$$[0, 1, 0, 2, 4, 0, 3, 0] \tag{1}$$

3.2 Initialization

Individuals were initialized by shuffling a list of all the patients, and assigning them round-robin to the nurses. This semi-random approach ensured that the patients were evenly distributed, and also allowed for a significant aspect of randomness. A clustering approach was also attempted, where the patients were grouped by using k-means clustering, however this approach turned out to provide low diversity.

3.3 Fitness and penalty

The fitness calculation was straightforwards, the total travel time was calculated using the values in the travel times matrix. As for the penalty, a simple numeric sum of violations was multiplied with a penalty parameter, see equation 2. The excess strain was how much a nurses capacity had been exceeded, and time violations were both the missed time windows and late return time. This approach was very simple, but nonetheless proved to be very effective for producing optimal solutions.

$$fitness = sumTravelTimes + penalty \cdot \sum_{i=0}^{n}(excessStrain_i + timeViolation_i) \tag{2}$$

3.4 Parent selection

For parent selection a simple tournament selection was chosen, with a parameter k to determine the amount of individuals selected. Two groups were formed randomly, of k individuals each, and for each group the individual with the lowest fitness was chosen as a parent.

3.5 Crossover

Crossover was implemented using a simple Order 1 crossover as covered in a lecture by Mengshoel 2026b. To account for the duplicate zeros as a result of the representation, the amount of allowed zeros was counted to ensure the children had the same amount of zeros as the parents. The first and last zero were not included in the crossover operation, as this would produce invalid representations.

3.6 Mutation

Three different types of mutations were implemented, and the choice of which one to use was random. Only one mutation was applied for any individual. The first was a reversal mutation, where a random full route was chosen and reversed. This mutation had a 30% chance to occur. Next, a relocation mutation could occur with a 40% chance. This mutation moved a patient from one route to another. This operation could not move a patient to an empty route, which was done to promote exploitation of existing routes. Finally, a exchange mutation occurred with a 30% chance. This mutation simply swapped two patients in the gene, and worked across routes.

3.7 Self-adaptive crowding

The algorithm used generalized crowding with a self-adaptive scaling factor, as covered in lecture 7 by Bergquist 2026. A scaling factor was randomly set for each individual during initialization, with a parameter providing the ceiling of the scaling factor. During crossover the scaling factors were averaged, and during mutation the scaling factor drifted with a Gaussian function. For each pair during the croding step of the algorithm, the scaling factor of the most fit individual (lowest fitness in this case) was used for the replacement probability. Cosine similarity was used to compute similarity, see equation 3. Other similarities were tried, but these generally were too computationally expensive.

$$\text{cosineSimilarity}(\mathbf{a}, \mathbf{b}) = \frac{\sum_i a_i b_i}{\sqrt{\sum_i a_i^2} \cdot \sqrt{\sum_i b_i^2}} \tag{3}$$

3.8 Elite selection

For each generation, the most fit individuals were chosen and copied over into the new population. Preference was given to legal solutions, but if there were not enough legal solutions the most fit illegal ones were used to fill up the remaining places. The amount of elites was controlled by a parameter.

3.9 Entropy

Edge entropy was used in order to give some idea of diversity in the population. From the groups research this was not a perfect measure of diversity, as this method does not account for the route structure of the problem. For this reason, it was used somewhat sparingly. However, it gave some idea of the change in diversity for different approaches. The entropy was calculated as shown in equation 4.

$$\begin{aligned} H &= -\sum_{e \in E} p(e) log_2 p(e) \\ p(e) &= \frac{c(e)}{\sum_{e' \in E} c(e')} \end{aligned} \tag{4}$$

Where E is the set of unique edges, and c(e) is the count of many times the edge appears across all individuals, and $\sum_{e' \in E} c(e')$ is the total number of edges across all individuals.

3.10 Islands

Early iterations of the algorithm, before the penalty was applied, showed that it was capable of producing individuals with very low travel times. However, when the penalty was applied these individuals were not able to develop. For this reason, the group decided to try a hierarchical approach, where several islands were run with a very low penalty in order to get individuals with low travel times. These islands also had a high crossover rate and low mutation, preferring exploration over exploitation. After this first stage, the best individuals from each island were combined into a new population for a second stage. This stage had a much higher penalty, and was set with parameters like higher k for the tournament selection and elite selection to cause an extreme selection pressure. Figure 1 shows the general structure of the approach. After running this stage for a certain number of generations, the most fit legal solution was returned as the solution. The idea behind this approach was to encourage high amounts of exploration and high diversity in the first stage, before boiling the individuals down to an optimal legal solution in the second stage. Since the first stage of islands ran separately, multi-threading was implemented to improve performance. The results section documents the solutions found using this approach.

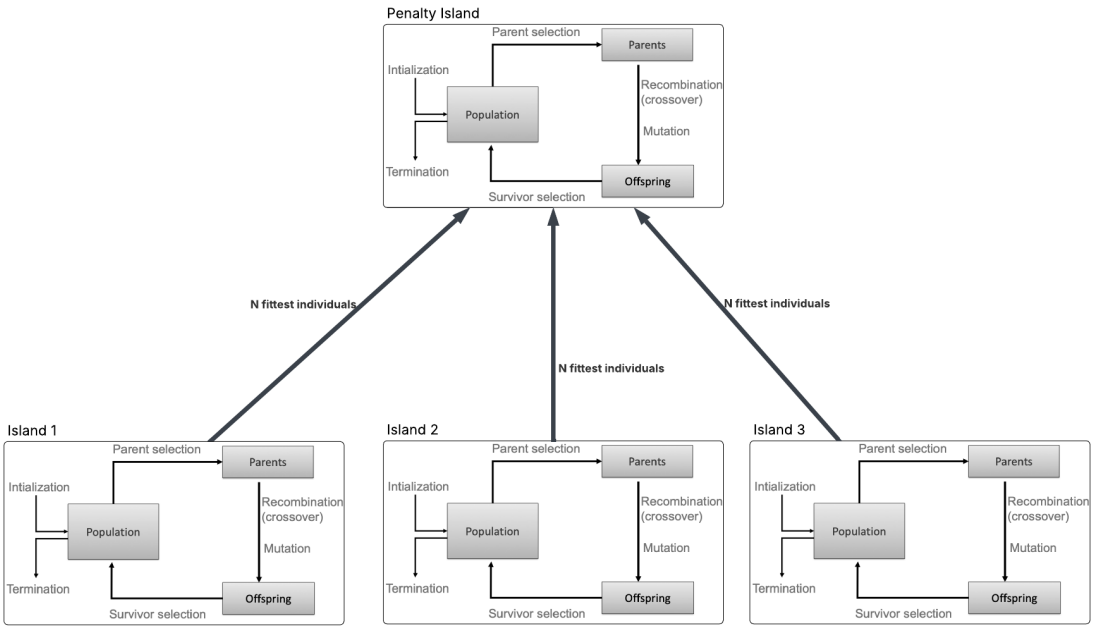

Figure 1: Penalty Island Diagram Source: Mengshoel 2026a

4 Results

We were able to achieve the 5% margin for all the test instances. All the runs were performed with a randomized seed, which is provided along with the parameters for reproducibility.

4.1 Test Instance 1

Running the algorithm described above gave a solution for this instance of 1569.77, which is 3.77% above the provided benchmark. The parameters used for this instance are listed in Table 1.

Parameter	Value
Stage 1	
Seed	1363486608
numIslands	5
islandPopSize	1000
stage1Gens	3000
elitesPerIsland	200
kParents	3
penalty	0.05
crossoverRate	0.9
mutationRate	0.3
scalingFactor	2.5
kElites	6
Stage 2	
exploitPopSize	1000
exploitKParents	20
exploitPenalty	2.5
exploitGens	3000
exploitCrossover	0.5
exploitMutation	0.9
exploitKElites	20

Table 1: Instance 1 Parameters

The solution had 13 assigned routes, with the rest of the nurses being unassigned. As seen in Figure 2, the routes seems to be somewhat grouped together. This is in line with what we would expect for a solution to such a problem. There are some crossings, but mostly the routes are separated.

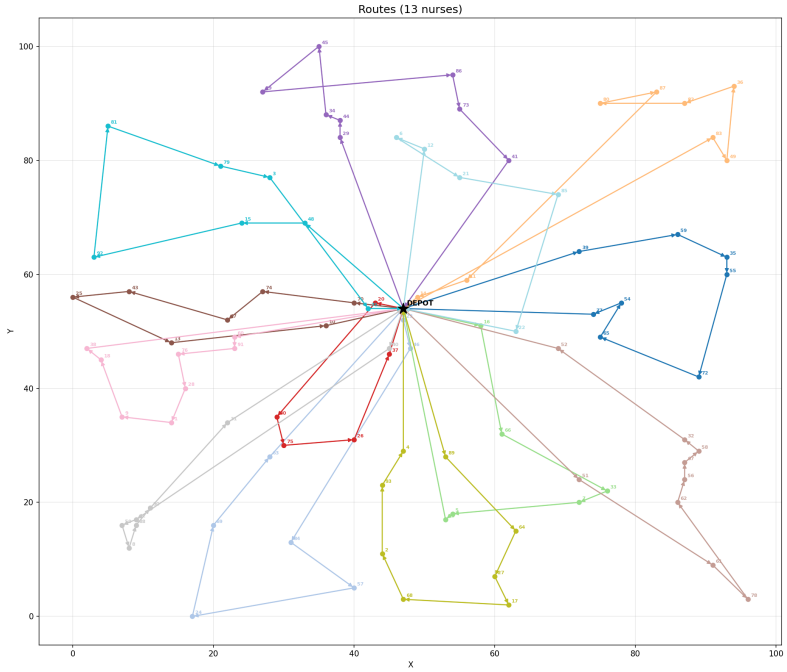

Figure 2: Instance 1 solution

4.2 Test Instance 2

For this instance we achieved a travel time of 1657.27, which is 4.04% above the benchmark. Table 2 below shows the parameters used.

Parameter	Value
Stage 1	
Seed	1891472797
numIslands	5
islandPopSize	1000
stage1Gens	3000
elitesPerIsland	200
kParents	3
penalty	0.05
crossoverRate	0.9
mutationRate	0.3
scalingFactor	2.5
kElites	6
Stage 2	
exploitPopSize	1000
exploitKParents	20
exploitPenalty	2.5
exploitGens	3000
exploitCrossover	0.5
exploitMutation	0.5
exploitKElites	20

Table 2: Instance 2 parameters

This solution has 14 assigned routes, shown in Figure 3, with the rest being unassigned. As with the previous solution, the routes here are grouped fairly evenly.

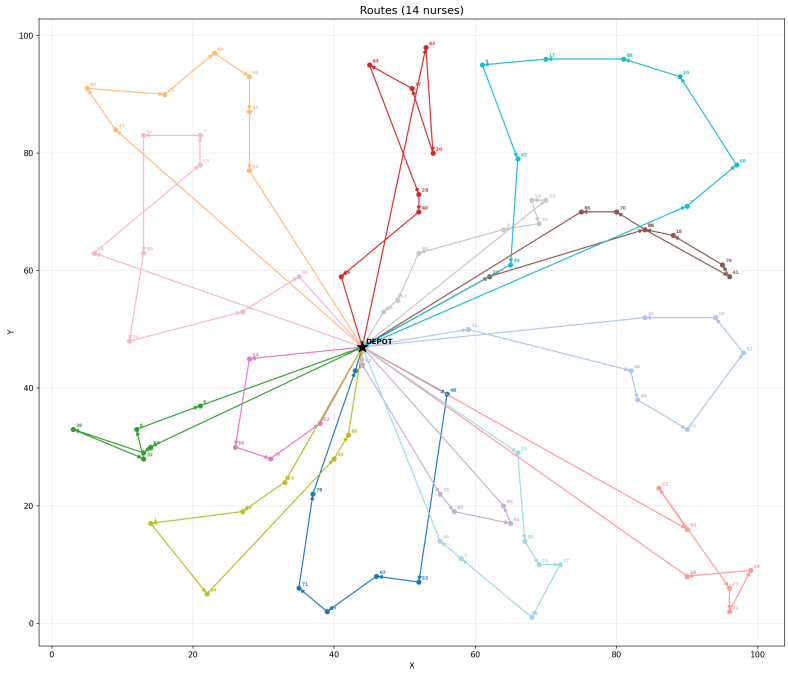

Figure 3: Instance 2 solution

4.3 Test instance 3

Finally, instance 3 was solved with a travel time of 2358.22, 2.45% above the benchmark. See Table 3 for the parameter setup.

Parameter	Value
Stage 1	
Seed	191483616
numIslands	5
islandPopSize	1000
stage1Gens	3000
elitesPerIsland	200
epsilon	0.1
kParents	3
penalty	0.05
crossoverRate	0.9
mutationRate	0.3
scalingFactor	2.5
kElites	6
Stage 2	
exploitPopSize	1000
exploitKParents	20
exploitPenalty	2.5
exploitGens	3000
exploitCrossover	0.5
exploitMutation	0.5
exploitKElites	20

Table 3: Instance 3 Parameters

Figure 4 shows the plot of the 19 different routes. This plot is a bit different, as there seems to be much more overlap between routes.

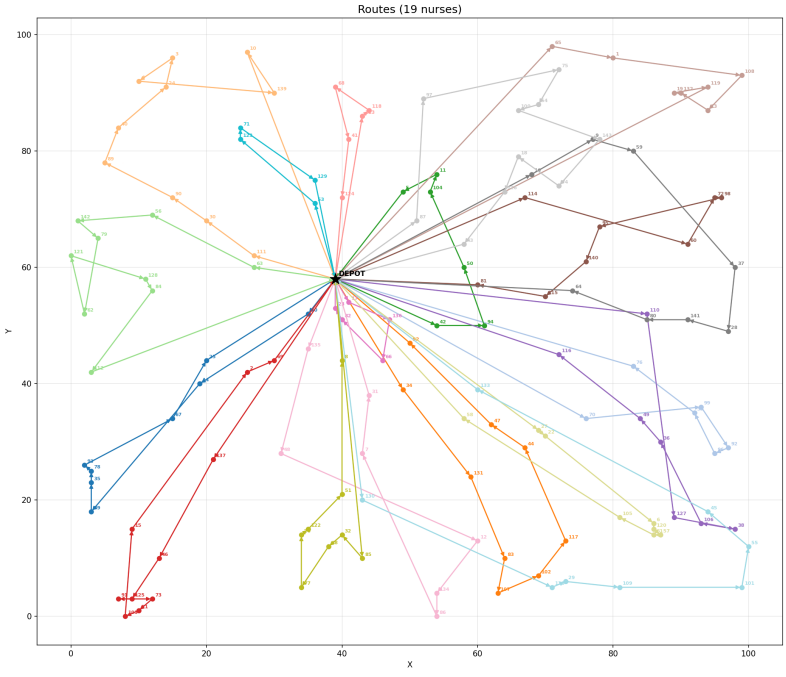

Figure 4: Instance 3 solution

5 Conclusion

The study presents a genetic algorithm designed to solve a vehicle routing problem in which nurses must visit home care patients within specific time windows while respecting capacity and scheduling constraints. A flat integer representation with depot markers was used to encode routes, allowing flexible route structures and efficient travel time calculations. The algorithm relied on a ruin and repair style search combined with crossover, mutation, and tournament selection to evolve solutions that minimize total travel time. Constraint violations were handled through a penalty term added to the fitness value. While this representation required additional checks to prevent malformed solutions, it allowed the algorithm to explore a wide range of routing configurations.

A key contribution of the work is the hierarchical island strategy that separates exploration from constraint satisfaction. In the first stage several islands evolved solutions under a low penalty, encouraging the discovery of routes with very short travel times even if they were temporarily infeasible. The best individuals from these islands were then combined in a second stage with stronger penalties and selection pressure, guiding the population toward feasible solutions. This approach consistently produced results within five percent of the benchmark across all test instances, showing that a staged evolutionary process can effectively balance exploration, diversity, and feasibility in complex routing problems. The method required longer runtimes and slower parameter tuning, yet the improvement in solution quality made this tradeoff worthwhile.

References

Bergquist, Jon Å. (2026). *Parameters setting, Tuning, and Control. Feedback Project 1 and tips for future projects. IT3708 BioAI.* Available at: https://ntnu.blackboard.com. Accessed: 8 March 2026.

Mengshoel, Ole J. (2026a). *Evolutionary Computing. IT3708 Bio-Inspired Artificial Intelligence.* Available at: https://ntnu.blackboard.com. Accessed: 8 March 2026.

Mengshoel, Ole J. (2026b). *Permutation representations & Constraint handling + minor tips for Project 2. IT3708 BioAI.* Available at: https://ntnu.blackboard.com. Accessed: 8 March 2026.